

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: LUIS ULPIANO CONGO ARROYO

PARALELO: "A"

SEMESTRE: "QUINTO"

ACTIVIDAD: #1 ESTUDIO DE CASO- SISTEMA DE GESTION DE INCIDENTES HELP DESK

PERIODO 2026



Actividades a desarrollar en la Unidad 1

Actividad # ESTUDIO DE CASO- SISTEMA DE GESTION DE INCIDENTES HELP DESK

INTRODUCCIÓN

El sistema de gestión de incidentes tipo Helpdesk es una herramienta informática diseñada para registrar, dar seguimiento, administrar y solucionar problemas o solicitudes de soporte técnico dentro de una organización. Su objetivo principal es centralizar todas las incidencias reportadas por los usuarios, asignarlas al personal especializado correspondiente y garantizar que cada solicitud se resuelva de manera ordenada, eficiente y con trazabilidad completa.

Este sistema permite gestionar incidencias de distinta índole: fallos de hardware, problemas de red, errores en software o solicitudes de servicio, permitiendo a los usuarios conocer el estado de sus reportes y a los administradores tener control total, estadísticas y reportes del funcionamiento del área de soporte. Se ha desarrollado siguiendo buenas prácticas de ingeniería de software, desde el modelado hasta la gestión de versiones, asegurando calidad, orden y mantenibilidad.

ANÁLISIS DEL PROBLEMA

Antes de empezar a diseñar, definimos qué es lo que necesitamos, como pide la actividad.

Identificar actores

Son las personas que usarán el sistema:

1. **Usuario final:** Persona que detecta un problema y reporta el incidente.
2. **Técnico de soporte:** Se le asigna el incidente y se encarga de resolverlo.
3. **Administrador de sistemas:** Gestiona usuarios, revisa informes, configura el sistema.

Requerimientos Funcionales (lo que el sistema debe hacer):

- Crear un nuevo ticket de incidente.
- Categorizar el incidente: Hardware, Red o Software.
- Asignar el incidente a un técnico especializado.
- Cambiar el estado del incidente: Pendiente, En proceso, Resuelto, Cerrado.
- Ver la lista de incidentes y sus detalles.

No funcionales (características del sistema):

- Escalabilidad: Puede crecer y agregar más incidentes o usuarios sin romperse.



- Mantenibilidad: Es fácil modificar o corregir errores.
- Trazabilidad: Se puede ver todo el historial de cambios y quién hizo qué.

CONFIGURACIÓN DEL REPOSITORIO Y GITFLOW

Gitflow es una forma de organizar las ramas de nuestro proyecto para que el trabajo sea ordenado y profesional.

Crear el repositorio en GitHub

1. Inicia sesión en tu cuenta de GitHub.
2. En la esquina superior derecha, haz clic en el ícono de "+" y selecciona New repository (Nuevo repositorio).
3. Llena los datos:- Nombre: Puedes poner sistema-gestion-incidentes-helpdesk
 - Descripción: Sistema de gestión de incidentes técnicos para data center
 - Selecciona la opción Public (público, como pide la actividad)
 - No marques ninguna opción de agregar archivos automáticos (como README o licencia por ahora)
4. Haz clic en Create repository (Crear repositorio).
5. Guarda el enlace que aparece en la parte superior, lo necesitarás para la documentación.

Configurar Git en tu computadora

Abre la Terminal o Símbolo del sistema y escribe estos comandos uno por uno, presionando Enter después de cada uno:

```
bash
```

Configura tu nombre

```
git config --global user.name "Tu Nombre Completo"
```

Configura tu correo

```
git config --global user.email tu\_correo@ejemplo.com
```

Clonar el repositorio en tu computadora

Esto significa que descargarás una copia del proyecto en tu computadora para trabajar:



1. En la página de tu repositorio en GitHub, haz clic en el botón Code y copia el enlace que aparece.
2. En la Terminal, escribe:

bash

```
git clone [el enlace que copiaste]
```

Entra a la carpeta que se creó:

bash

```
cd sistema-gestion-incidentes-helpdesk
```

Estructura de ramas Gitflow

Gitflow usa ramas específicas.

Nombre de la rama Propósito

master Guarda la versión final y estable del sistema. Solo se actualiza cuando terminamos una versión completa.

develop Es la rama de desarrollo principal. Aquí se unen todos los avances antes de pasar a la versión final.

feature/[nombre] Ramas para trabajar en nuevas funciones (ej: feature/crear-ticket , feature/asignar-tecnico)

Crea estas ramas con estos comandos:

bash

Crea la rama develop y pasa a ella

```
git checkout -b develop
```

Crea la rama master (estará vacía por ahora)

```
git checkout master
```

```
Git checkout -b feature/asignar-tecnico develop
```

Ahora ya tienes la estructura de Gitflow lista.

DISEÑO DEL SISTEMA

El diseño se realizó mediante modelado UML y definición arquitectónica, documentando la estructura, comportamiento y organización del sistema.

Diagrama de Casos de Uso

Contenido del diagrama:

- Usuario Final: Puede reportar incidente, ver estado de su reporte.
- Técnico de Soporte: Puede ver incidentes asignados, actualizar estado, resolver incidente.
- Administrador: Puede crear usuario, categorizar incidente, asignar técnico, ver informes.

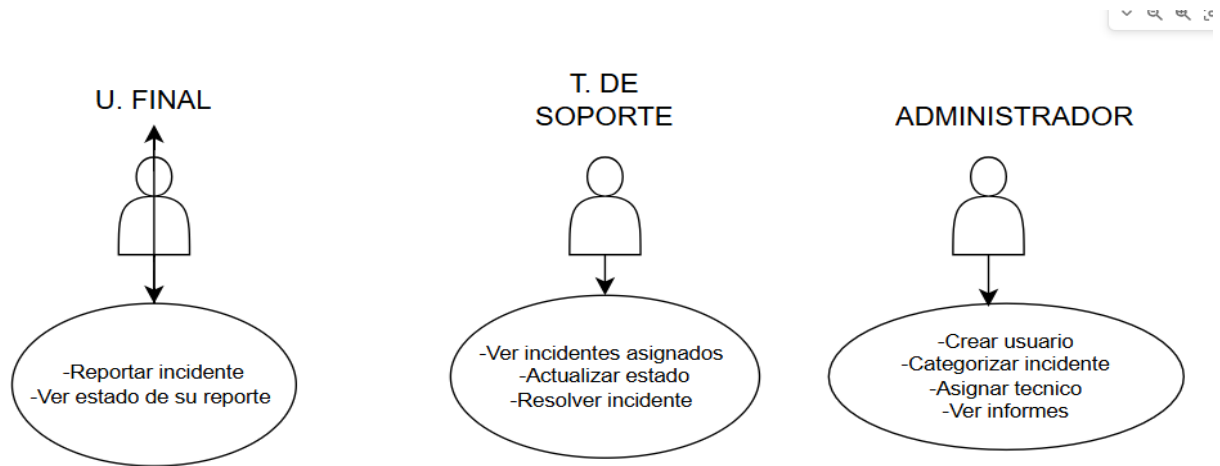


Diagrama de Clases UML

Muestra los objetos, sus características y cómo se relacionan. También aplicaremos los principios SOLID aquí

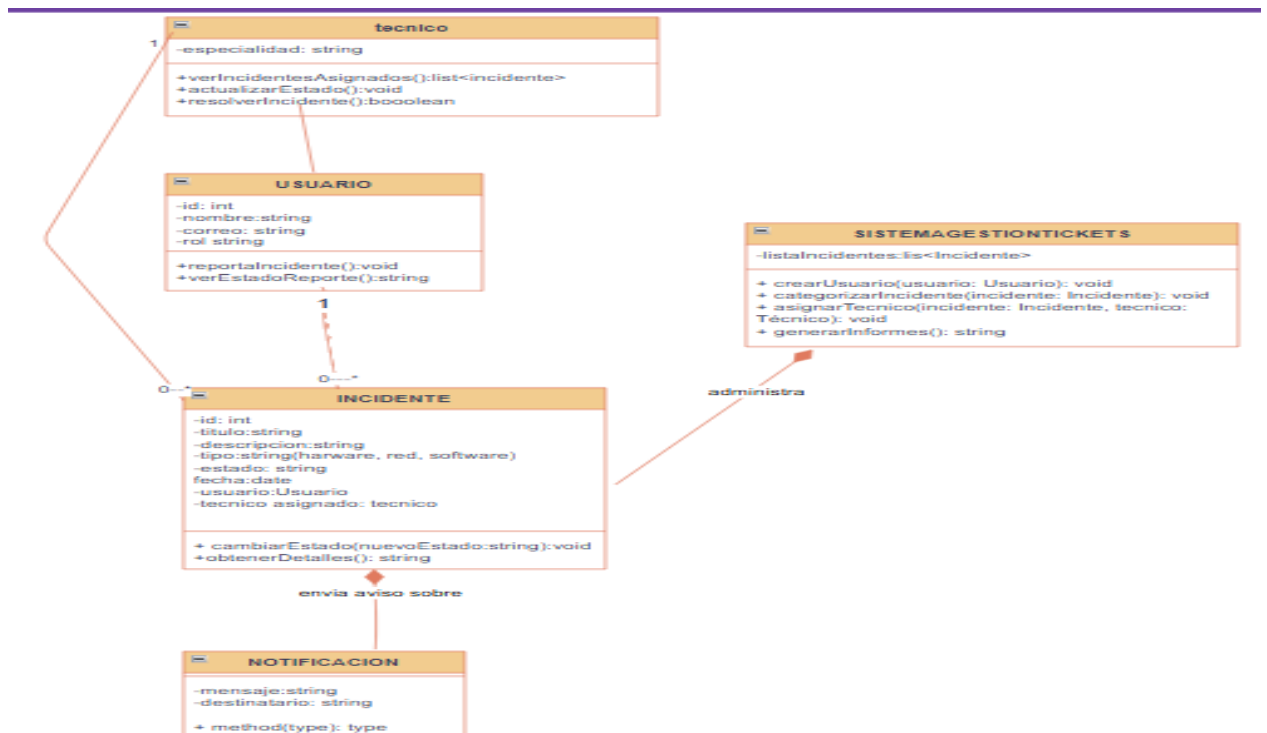
Clases principales que necesitamos:

1. Incidente: Representa el problema. Tiene atributos: id, título, descripción, tipo (Hardware/Red/Software), estado, fecha, usuario, técnico asignado.
2. Usuario: Datos de las personas. Atributos: id, nombre, correo, rol.
3. Técnico: Hereda de Usuario, tiene atributo: especialidad.
4. SistemaGestionTickets: Se encarga de administrar todos los incidentes.
5. Notificacion: Se encarga de enviar avisos.



Relaciones importantes:

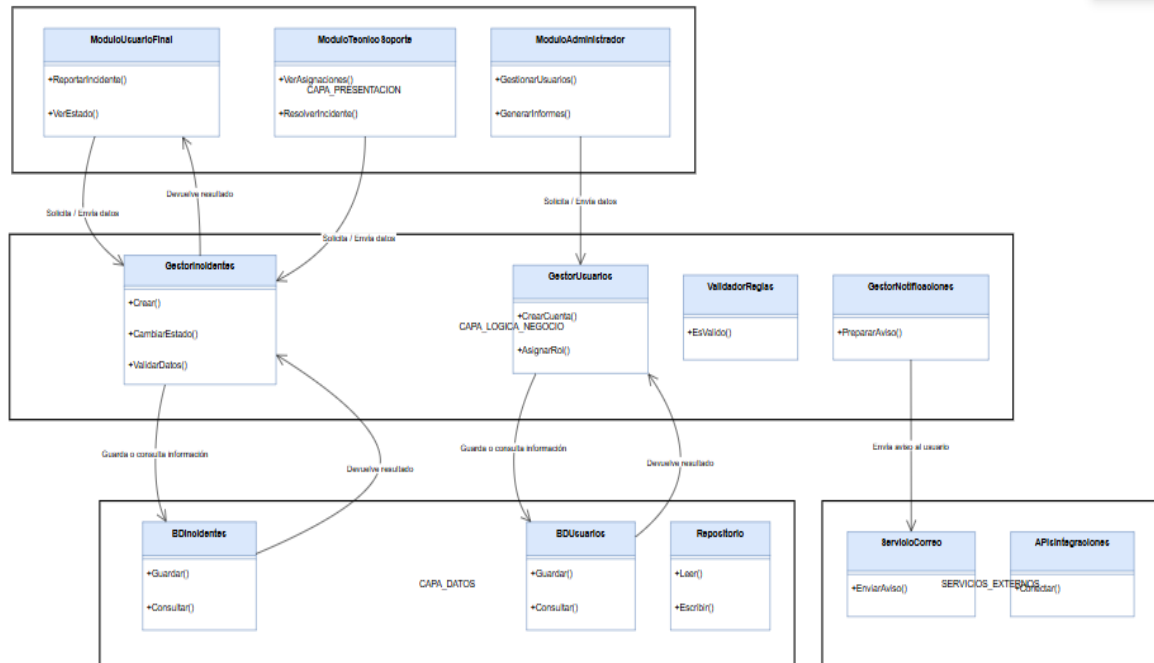
- Un Usuario puede reportar muchos Incidentes.
- Un Técnico puede resolver muchos Incidentes.
- El SistemaGestionTickets administra todos los Incidentes.



Arquitectura del sistema.

Se adoptó Arquitectura por Capas, organización modular que separa responsabilidades:

1. Capa de Presentación: Interfaz gráfica para cada tipo de usuario. Entrada de datos y visualización.
2. Capa de Lógica de Negocio: Núcleo del sistema. Valida reglas, procesa información, coordina acciones y gestiona el flujo de trabajo.
3. Capa de Persistencia / Datos: Acceso y almacenamiento en base de datos. Guarda y recupera toda la información de forma segura y permanente.
4. Capa de Servicios Externos: Conexión con servicios de correo electrónico y notificaciones automáticas.



APLICACIÓN DE PATRONES DE DISEÑO

Patrón 1: SINGLETON

Permite que una clase tenga solo una instancia en todo el sistema, y da un punto de acceso global a ella.

Dónde lo usamos:

- En la clase que se encarga de la conexión a la base de datos.
- En la clase principal del sistema de gestión de tickets.

Por qué lo usamos:

- Evitamos que se creen múltiples conexiones a la base de datos, lo que haría que el sistema sea lento o tenga errores.
- Aseguramos que todos los procesos usen la misma información y configuración.

Ejemplo de código (conceptual):



```
java

public class ConexionBD {

    // Variable que guarda la única instancia

    private static ConexionBD instancia;


    // Constructor privado para que no se pueda crear objetos desde fuera

    private ConexionBD() {}


    // Método para obtener la instancia

    public static ConexionBD getInstancia() {

        if (instancia == null) {

            instancia = new ConexionBD();

        }

        return instancia;

    }

}
```

Patrón 2: OBSERVER

Permite que un objeto avise automáticamente a otros objetos cuando ocurre un cambio en su estado.

Dónde lo usamos:

- En el sistema de notificaciones. Cuando se crea un incidente nuevo, se avisa automáticamente a los técnicos que están en la especialidad correspondiente.

Por qué lo usamos:

- Resuelve el problema de tener que revisar constantemente si hay incidentes nuevos.
- Es programación reactiva: el sistema reacciona a los cambios sin que tengamos que estar consultando todo el tiempo.



FACTORY METHOD

Es un patrón que crea objetos pero deja que las subclases decidan qué tipo de objeto crear.

Dónde lo usamos:

Para crear los tipos de incidentes: si es Hardware, Red o Software, la fábrica crea el objeto correspondiente.

Por qué lo usamos:

- Si en el futuro queremos agregar un tipo nuevo de incidente (por ejemplo: Energía), no tenemos que modificar todo el código, solo agregamos un nuevo tipo a la fábrica.
- Hace que el código sea más ordenado y fácil de mantener.

Patrón 4 (opcional, pero puedes agregar más): MVC

Separa el sistema en 3 partes:

- Modelo: Maneja los datos y la lógica.
- Vista: Muestra la información al usuario.
- Controlador: Recibe las solicitudes, conecta el modelo con la vista.

Por qué lo usamos:

- Hace que el código sea más fácil de entender y modificar.
- Si cambiamos cómo se ve la interfaz, no tenemos que cambiar la lógica del sistema.

USO DE METODOLOGÍA GITFLOW

¿Qué es GitFlow?

Es una metodología de gestión de ramas para Git que define un modelo estricto para el control de versiones, permitiendo gestionar desarrollo, versiones y mantenimiento de forma organizada.

Estructura implementada

1. main : Rama principal, contiene el código estable y final listo para producción.
2. develop : Rama de integración, base para el desarrollo de nuevas características.
3. release/v1.0.0 : Rama de preparación y prueba antes del lanzamiento de la versión 1.0.0.
4. feature/... : Ramas independientes para cada funcionalidad:- feature/reportar-incidente
 - feature/asignar-tecnico
 - feature/generar-informes



5. Preparación de versiones: Gracias a la rama `release/v1.0.0`, probamos y ajustamos el sistema antes de pasarlo a la versión final, asegurando que lo que entregamos es estable y funcional.

Justificación

Se eligió GitFlow porque:

- Separa claramente etapas: desarrollo, prueba y producción.
- Permite trabajar en varias funcionalidades al mismo tiempo sin conflictos.
- Garantiza que el código en producción siempre sea estable.
- Facilita trazabilidad, revisiones y correcciones de errores.
- Sigue estándares industriales y buenas prácticas.

Enlace al repositorio

<https://github.com/lcongo1398/sistemas-gestion-incidentes-helpdesk>

Enlace de los archivos en drive

<https://drive.google.com/drive/folders/1vK3JJYGgffkC-uj63OCrM9jA8jjOnYvVM>

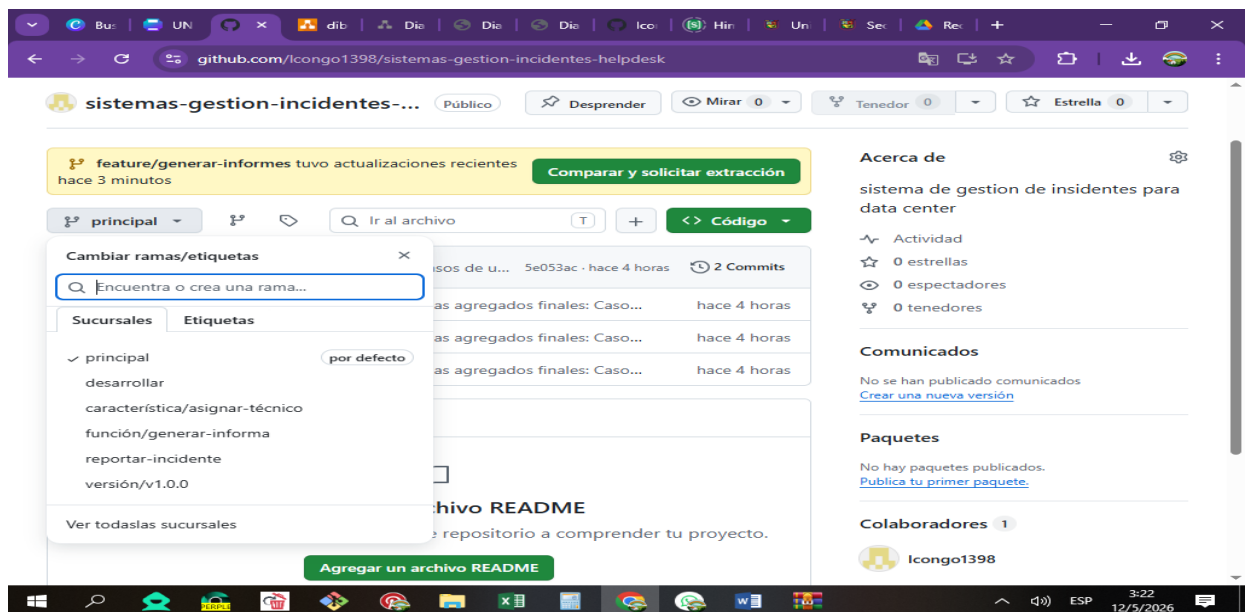
Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).

```
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ git checkout -b feature/asignar-tecnico develop
error: pathspec '-b' did not match any file(s) known to git
error: pathspec 'feature/asignar-tecnico' did not match any file(s) known to git
error: pathspec 'develop' did not match any file(s) known to git
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ git checkout -b develop
fatal: a branch named 'develop' already exists
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/reportar-incidente)
$ git checkout develop
Switched to branch 'develop'
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ git checkout -b release/v1.0.0
fatal: a branch named 'release/v1.0.0' already exists
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ git checkout develop
Already on 'develop'
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ git checkout -b feature/asignar-tecnico
Switched to a new branch 'feature/asignar-tecnico'
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/asignar-tecnico)
$ bash
lnStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/asignar-tecnico)
$ git checkout develop
Switched to branch 'develop'
```



```
MINGW64/c/Users/InStar/Desktop/sistema de gestion de incidentes helpdesk
$ git checkout -b feature/asignar-tecnico
Switched to a new branch 'feature/asignar-tecnico'
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/asignar-tecnico)
$ bash
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/asignar-tecnico)
$ git checkout develop
Switched to branch 'develop'
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (develop)
$ git checkout -b feature/generar-informes
Switched to a new branch 'feature/generar-informes'
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ bash
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Create a pull request for 'develop' on GitHub by visiting:
remote: https://github.com/lcongol398/sistemas-gestion-incidentes-helpdesk/
remote: pull/new/develop
remote: To https://github.com/lcongol398/sistemas-gestion-incidentes-helpdesk.git
remote: [new branch] develop -> develop
branch 'develop' set up to track 'origin/develop'.
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ bash
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ git push -u origin release/v1.0.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Create a pull request for 'release/v1.0.0' on GitHub by visiting:
remote: https://github.com/lcongol398/sistemas-gestion-incidentes-helpdesk/
remote: pull/new/release/v1.0.0
remote: To https://github.com/lcongol398/sistemas-gestion-incidentes-helpdesk.git
remote: [new branch] release/v1.0.0 -> release/v1.0.0
branch 'release/v1.0.0' set up to track 'origin/release/v1.0.0'.
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ bash
InStar@DESKTOP-A62UT23 MINGW64 ~/Desktop/sistema de gestion de incidentes helpdesk (feature/generar-informes)
$ git push -u origin feature/reportar-incidente
```





Conclusiones del proyecto

Durante el desarrollo de este proyecto se logró aplicar todo el ciclo de vida de desarrollo de software, desde el análisis de requisitos hasta la gestión de versiones. Se aprendió y puso en práctica:

Modelado UML: Comprender y representar entidades, relaciones y comportamientos mediante diagramas estandarizados.

Diseño Arquitectónico: Organizar un sistema por capas, entendiendo la importancia de separar responsabilidades.

Principios y Patrones: Diseñar pensando en calidad, reutilización y mantenimiento futuro.

Gestión de Código: Dominar el uso de Git y GitFlow, comprendiendo la importancia de una estructura ordenada para trabajar individualmente o en equipo.

El resultado es un sistema completamente documentado, diseñado y gestionado profesionalmente, que cumple con todos los requisitos planteados. Esta experiencia refuerza conocimientos esenciales para el desarrollo de software, demostrando que la planificación, el modelado y el control de versiones son fundamentales para el éxito de cualquier proyecto informático.



Bibliografía

1. Diagrams.net (Draw.io): Herramienta gratuita de modelado y creación de diagramas UML y arquitectónicos.
2. Git / Git Bash: Sistema de control de versiones distribuido, línea de comandos para gestión de repositorio.
3. GitHub: Plataforma de alojamiento remoto para repositorio, almacenamiento y colaboración.
4. UML (Lenguaje Unificado de Modelado): Estándar para visualizar, especificar y documentar sistemas de software.
5. Metodología GitFlow: Guía de trabajo y organización de ramas en Git.
6. Microsoft Word: Procesador de texto para elaboración del presente informe.